

Load Data into a DynamoDB Table

Melvin green

Table: ContentCatalog - Items returned (6) Actions

Scan started on August 04, 2026, 21:32:57

☐	Id (Number)	Authors	ContentType	Difficulty	Price	ProjectCategory	Published	Score
☐	1	[{"S": "Nat...}]	Project	Easy peasy	0	Storage	true	
☐	2	[{"S": "Ne...}]	Project	Easy peasy	0	Analytics	true	
☐	3	[{"S": "Ne...}]	Project	Easy peasy	0	AI/ML	true	
☐	201		Video		0			[...]
☐	202		Video		0			[...]
☐	203		Video		0			[...]

Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a non-relational database service. It is useful because non-relational databases use structures other than rows and columns to organize data. It also lets me enter data directly from the console.

How I used Amazon DynamoDB in this project

Here's a corrected and slightly expanded version: "In today's project, I used Amazon DynamoDB to create tables using CloudShell, load data into them via the AWS CLI, and edit that data directly within my DynamoDB tables. I also explored how DynamoDB's flexible, non-relational structure and partition keys make it a fast and scalable choice for storing content with varying attributes.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is how efficient CloudShell could be when it came to creating, inserting, and deleting tables.

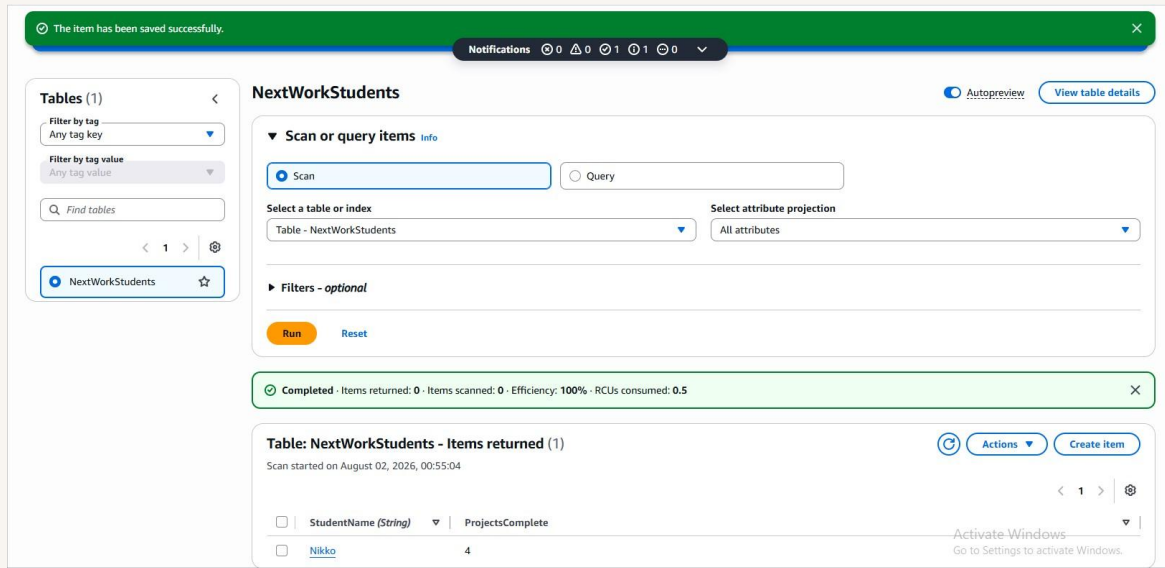
This project took me...

This project took me 1 hour to complete a document.

Create a DynamoDB table

DynamoDB tables organises data using...

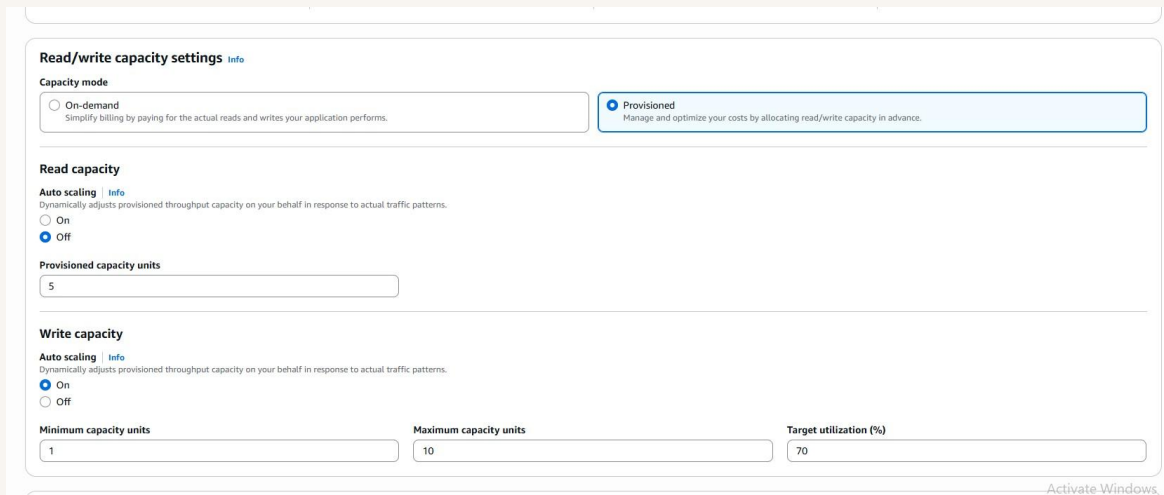
An attribute is a piece of data about an item. In this case, our item is Nikko, and the attribute is the number of projects Nikko has completed.



Read and Write Capacity

Read capacity units (RCUs) and write capacity units (WCUs) are ways to measure how much throughput DynamoDB allocates to read and write operations. One read capacity unit (RCU) allows one strongly consistent read per second, or two eventually consistent reads per second, for an item up to 4 KB in size. Write capacity units (WCUs) work similarly. One WCU allows one write per second for an item up to 1 KB in size, letting you add, update, or delete data.

Amazon DynamoDB's Free Tier covers a limited amount of provisioned capacity. I turned off auto scaling because auto scaling can automatically adjust your database's performance, which could cause you to exceed the Free Tier limits.



The screenshot shows the 'Read/write capacity settings' page in the AWS Management Console. It is divided into three main sections: 'Capacity mode', 'Read capacity', and 'Write capacity'. In the 'Capacity mode' section, the 'Provisioned' option is selected, indicating that capacity is managed in advance. The 'Read capacity' section has 'Auto scaling' turned off, and the 'Provisioned capacity units' is set to 5. The 'Write capacity' section also has 'Auto scaling' turned off, with 'Minimum capacity units' set to 1, 'Maximum capacity units' set to 10, and 'Target utilization (%)' set to 70. An 'Activate Windows' watermark is visible in the bottom right corner of the console window.

Read/write capacity settings [Info](#)

Capacity mode

☐ On-demand
Simplify billing by paying for the actual reads and writes your application performs.

☒ **Provisioned**
Manage and optimize your costs by allocating read/write capacity in advance.

Read capacity

Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☐ On

☒ **Off**

Provisioned capacity units

5

Write capacity

Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☒ **On**

☐ Off

Minimum capacity units

1

Maximum capacity units

10

Target utilization (%)

70

Activate Windows

Using CLI and CloudShell

AWS CloudShell is a shell in my AWS Management Console, which gives me the ability to run code.

AWS CLI gives me the ability to run code and is the software that lets me create, delete, and update AWS resources with commands instead of clicking through my console

I ran a CLI command in AWS CloudShell that created 4 new tables in AWS DynamoDB, each with specific attributes and settings.

```

CloudShell
us-east-2 +
~
$ aws dynamodb create-table \
  --table-name ContentCatalog \
  --attribute-definitions \
  AttributeDefinitions={AttributeType=N} \
  --key-scheme \
  AttributeKeyScheme={AttributeName=Id,KeyType=HASH} \
  --provisioned-throughput \
  ReadCapacityUnits=1,WriteCapacityUnits=1 \
  --query "TableDescription.TableStatus" \
  --output json
{"TableStatus": "CREATING"}
$ aws dynamodb create-table \
  --table-name Forum \
  --attribute-definitions \
  AttributeDefinitions={AttributeType=S} \
  --key-scheme \
  AttributeKeyScheme={AttributeName=Name,AttributeType=S} \
  --provisioned-throughput \
  ReadCapacityUnits=1,WriteCapacityUnits=1 \
  --query "TableDescription.TableStatus" \
  --output json
{"TableStatus": "CREATING"}
$ aws dynamodb create-table \
  --table-name Post \
  --attribute-definitions \
  AttributeDefinitions={AttributeType=S} \
  --key-scheme \
  AttributeKeyScheme={AttributeName=Subject,AttributeType=S} \
  --provisioned-throughput \
  ReadCapacityUnits=1,WriteCapacityUnits=1 \
  --query "TableDescription.TableStatus" \
  --output json
{"TableStatus": "CREATING"}
$ aws dynamodb create-table \
  --table-name Comment \
  --attribute-definitions \
  AttributeDefinitions={AttributeType=S} \
  --key-scheme \
  AttributeKeyScheme={AttributeName=CommentText,AttributeType=S} \
  --provisioned-throughput \
  ReadCapacityUnits=1,WriteCapacityUnits=1 \
  --query "TableDescription.TableStatus" \
  --output json
{"TableStatus": "CREATING"}
$

```

Loading Data with CLI

I ran a CLI command in AWS CloudShell using `aws dynamodb batch-write-item`, which is used to load or insert data. The `--request-items` flag tells DynamoDB that the items are stored inside a file that it needs to retrieve. Finally, `file://` tells DynamoDB that the file is stored locally in the CloudShell environment, with the name `FILENAME.json`.

```

CloudShell
us-east-2 +
~
$ cat ContentCatalog.json
{
  "Forum": [
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "I have a question"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "2"},
          "Comments": {"N": "4"},
          "Views": {"N": "1000"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "Events"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "1"}
        }
      }
    }
  ]
}
$ aws dynamodb batch-write-item --request-items file://ContentCatalog.json
{"UnprocessedItems": {}}
$ cat Forum.json
{
  "Forum": [
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "I have a question"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "2"},
          "Comments": {"N": "4"},
          "Views": {"N": "1000"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "Events"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "1"}
        }
      }
    }
  ]
}
$ aws dynamodb batch-write-item --request-items file://Forum.json
{"UnprocessedItems": {}}
$ cat Post.json
{
  "Post": [
    {
      "PutRequest": {
        "Item": {
          "Subject": {"S": "I have a question"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "2"},
          "Comments": {"N": "4"},
          "Views": {"N": "1000"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "Subject": {"S": "Events"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "1"}
        }
      }
    }
  ]
}
$ aws dynamodb batch-write-item --request-items file://Post.json
{"UnprocessedItems": {}}
$ cat Comment.json
{
  "Comment": [
    {
      "PutRequest": {
        "Item": {
          "CommentText": {"S": "I have a question"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "2"},
          "Comments": {"N": "4"},
          "Views": {"N": "1000"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "CommentText": {"S": "Events"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "1"}
        }
      }
    }
  ]
}
$ aws dynamodb batch-write-item --request-items file://Comment.json
{"UnprocessedItems": {}}
$ cat ContentCatalog.json
{
  "Forum": [
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "I have a question"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "2"},
          "Comments": {"N": "4"},
          "Views": {"N": "1000"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "Events"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "1"}
        }
      }
    }
  ]
}
$ aws dynamodb batch-write-item --request-items file://ContentCatalog.json
{"UnprocessedItems": {}}
$

```

Observing Item Attributes

Edit item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attribute name	Value	Type
Id - Partition key	1	Number
Authors	Insert a field	List
ContentType	Project	String
Difficulty	Easy peasy	String
Price	0	Number
ProjectCategory	Storage	String
Published	☒ True ☐ False	Boolean
Title	Host a Website on Amazon S3	String
URL	aws-host-a-website-on-s3	String

I checked a ContentCatalog item, which had the following attributes: Author, ContentType, Difficulty, Price, ProjectCategory, Published, Title, and URL

I checked another ContentCatalog item, which had a different set of attributes: ContentType, Price, Service, Title, URL, and VideoType.

Benefits of DynamoDB

Here's the corrected version: "A benefit of DynamoDB over relational databases is flexibility, because every item can have its own unique attributes, which is a huge

advantage. For instance, you could have three of the same product, except the colors are different. As well as speed,

Another benefit over relational databases is speed, because DynamoDB tables can use partition keys to split up a table and quickly find the items they're looking for.

Table: ContentCatalog - Items returned (6)

Scan started on August 04, 2026, 21:32:57

Actions

☐	Id (Number)	▲	Authors ▼	ContentType ▼	Difficulty ▼	Price ▼	ProjectCategory ▼	Published ▼	Size
☐	1		[{"S": "Nat...}]	Project	Easy peasy	0	Storage	true	
☐	2		[{"S": "Ne...}]	Project	Easy peasy	0	Analytics	true	
☐	3		[{"S": "Ne...}]	Project	Easy peasy	0	AI/ML	true	
☐	201			Video		0			[...]
☐	202			Video		0			[...]
☐	203			Video		0			[...]